The bunch of stuff after you type ls includes all the methods project can respond to. As you might imagine, that's rather a lot. All output in Pry goes through a paginator, though, so you can just type q to get back to the command line rather than wading through the entire listing.

If you start a command with a dot (.), Pry will consider that to be a shell command, so you can keep a Pry terminal open but still interact with, say, your Git repo.

And that's not all.

You can use the show-source command to display, in the Pry console, the source code of any method. You can request methods of the object in scope:

```
[1] pry(main)> project = Project.new(name: "Project Runway")
=> #<Project:0x007fcb8ff82700
 id: nil,
 name: "Project Runway",
 due_date: nil,
 created_at: nil,
 updated_at: nil,
 public: false
[2] pry(main)> cd project
[3] pry(#<Project>):1> show-source on_schedule?

From: /Users/noel/code/github/noelrappin/gatherer/app/models/project.rb @ line 52:
Owner: Project
Visibility: public
Number of lines: 5

def on_schedule?
  return false if projected_days_remaining.infinite?
  return false if projected_days_remaining.nan?
  (Date.today + projected_days_remaining) <= due_date
end
```

You can't see it here, but the method is syntax-colored in the console. You can do this for any method in the system using the ClassName#method_name syntax. Similarly, you can use the show-doc and ri methods to see just the documentation for a method you're interested in, which is particularly useful for Rails methods.

Pry has a couple of special variables that are always available. Like irb, the _ variable always refers to the result of the most recent expression evaluated. The variable _ex_ is the most recently raised exception, even if you've moved out of the exception scope. You can use cat —ex to view the code that caused the exception, and you can see the stack trace with _ex_.backtrace.